

mpc_cuda: Porting GMP/MPFR/MPC to CUDA

— Construction, Implementation Difficulties, and Performance —

tkouya

2026 年 6 月 11 日

概要

This report describes **mpc_cuda**, a port of the GNU multiple-precision stack—GMP, MPFR and MPC—that runs inside NVIDIA CUDA kernels. We cover the construction procedure, the implementation difficulties met during the port and their solutions, and a performance evaluation of basic linear algebra and elementary functions. We further introduce a compile-time fixed-precision, register-resident fast path, `cu_freal<PB>/cu_fcomplex<PB>`, and quantify the GPU/CPU performance crossover over the precision range 128–8192 bits. The primary hardware is the NVIDIA GB10 (compute capability 12.1, `sm_121`, CUDA 13); we additionally cross-compare the runtime-precision demos against an NVIDIA H100 NVL (compute capability 9.0, `sm_90`). All results are bit-identical to host MPFR/MPC (or within ≤ 1 ULP for the elementary functions).

目次

1	Introduction	1
2	Overall Design and Construction Procedure	2
2.1	The upstream sources	2
2.2	The transform-script approach	2
2.3	The layered phases	2
3	Implementation Difficulties and Solutions	3
3.1	Making it compile	3
3.2	Making it run correctly on the device	4
3.3	The hardest problem: device miscompile of <code>mpfr_div</code>	4
3.4	Stack, arena, and parallelism	5
3.5	Coexistence: the <code>cu_</code> namespace and the <code>umbrella</code> header	5
3.6	Other bugs and lessons	6

4	The Compile-Time Fixed-Precision Fast Path	6
4.1	Design	6
4.2	Elementary functions	6
5	Performance Evaluation	7
5.1	AXPY (real and complex)	7
5.2	Basic linear algebra (matrix-vector and matrix multiply)	8
5.3	Elementary functions	8
5.4	The fixed-precision fast path and a precision sweep	10
5.5	The fixed-precision path on the CPU	11
5.6	Discussion	12
5.7	Cross-platform comparison: GB10 vs H100 (synthesis)	12
6	Conclusion and Future Work	15

1 Introduction

Running arbitrary-precision arithmetic on the GPU is attractive for the numerical linear algebra of ill-conditioned problems and for high-precision simulation. Existing GPU multiple-precision libraries (e.g. CUMP ports GMP’s `mpf`) exist but often have loose rounding semantics and lack complex numbers and elementary functions. The goal of this project is to run the **correctly-rounded real semantics of MPFR**, and **the complex MPC layer on top of it**, inside CUDA kernels as **faithfully** as possible (a port of the upstream sources, not a reimplementa-tion). Concretely, the target was to evaluate $y = ax + y$ (AXPY) for both real (MPFR) and complex (MPC) operands on the GPU, match the CPU result bit-for-bit, and obtain a substantial speedup.

The report is organised as follows. Section 2 describes the overall design and construction procedure (the transform-script approach and the layered phases). Section 3 details the porting difficulties and their solutions. Section 4 presents the compile-time fixed-precision fast path, and Section 5 the performance evaluation (AXPY, linear algebra, elementary functions, and the precision sweep). Section 6 concludes.

2 Overall Design and Construction Procedure

2.1 The upstream sources

The base of the port is **mini-gmp** (`gmp-6.3.0/mini-gmp`), a subset implementation of GMP. `mini-gmp` provides, in a single C source, the core of `mpz` (multiple-precision integers)

and `mpn` (low-level limb arrays). MPFR-4.2.2 builds on `mini-gmp` with `--with-mini-gmp`; the few missing `mpn` routines (`mpn_divrem`, etc.) are supplied by MPFR’s own `mpfr-mini-gmp.c`. Thus **no full-GMP internals are required**. MPC-1.4.1 is the thin complex layer on top of MPFR and rides the same base.

2.2 The transform-script approach

Hand-forking the upstream sources makes it hard to re-apply upstream updates. Instead this project performs device adaptation through **re-runnable transform scripts** (`tools/cudafy_*.py`).

- `tools/cudafy_minigmp.py`: reads the pristine `mini-gmp.{c,h}` and emits `include/mpc_cuda/cuda_minigmp.h` and `src/cuda_minigmp.cu` with `__host__ __device__` prepended to every device-reachable function. Host-only functions (those using `stdio/realloc/ctype`) are detected by a word-boundary token scan.
- `tools/cudafy_mpfr.py`: applies the same adaptation to MPFR’s 263 sources.
- `tools/cudafy_mpc.py`: applies it to MPC’s 90 sources (reusing the MPFR machinery).

This keeps the upstream structure intact, so re-running the scripts re-applies upstream updates.

2.3 The layered phases

Development proceeded in four phases (plus 2b), each validated bit-for-bit against the host before building on it.

表1 The layered development phases

Phase	Layer	Content
1	mini-gmp	<code>mpz/mpn</code> on the device. <code>add/sub/mul/t-div_qr/pow_ui/get_str</code> bit-exact vs host (256 threads).
2	MPFR (faithful)	Real <code>mpfr-4.2.2</code> <code>init2/set/mul/add/get_d</code> run inside kernels.
2b	MPFR transcendentals	<code>pi/exp/log/sin/cos/atan/sqrt</code> work (requires the <code>mpfr_div</code> substitution below).
3	MPC (complex)	Real <code>mpc-1.4.1</code> <code>init2/set_d_d/mul/add</code> run.
4	AXPY demos / benchmarks	Real and complex AXPY, GPU vs CPU, time and accuracy.

3 Implementation Difficulties and Solutions

“It compiles” and “it runs correctly on the device” are different problems. This section classifies the difficulties that were the crux of the port.

3.1 Making it compile

1. **C11 _Noreturn**: rejected by nvcc’s C++ frontend. The host build’s `-DMPFR_HAVE_NORETURN` must be **dropped**.
2. **Inline assembly**: the `x86_64 umul_ppmm` etc. in `mpfr-longlong.h` are invalid on the device. `-DNO_ASM` selects the portable C fallbacks.
3. **No realloc**: CUDA device code has no `realloc`; it is emulated by `malloc+memcpy+free` (`mg_cuda_realloc`).
4. **C99 _Complex**: device code has no `_Complex`. MPC’s `mpc.h` `#if 1` (which defines `_MPC_HAVE_COMPLEX_H`) is turned into `#if 0`, disabling `mpc_set_dc/get_dc`.
5. **gmp.h shim**: MPC’s `mpc.h` `#includes gmp.h`. A shim pointing at the CUDA mini-gmp is generated, declaring minimal `gmp_randstate_t/mpf_t/mpq_t` stand-ins (absent from mini-gmp) so the prototypes parse.
6. **Token pasting**: MPC’s `set_x.c/set_x_x.c` expand `mpfr_set_##type` for all setters in one TU, pulling in `mpfr_set_f/_q` (absent under mini-gmp). Trapping device stubs are injected so the unit compiles.

3.2 Making it run correctly on the device

This is the substantively hard part: even when it compiles, host-oriented global state, function pointers, and optimisations misbehave on the device.

1. **Lookup tables**: file-scope `static const` arrays such as `invert_limb_table` and `__clz_tab` cannot be referenced from the device. They are tagged `MPFR_RODATA` (`__device__` in the device pass, empty in the host pass), so one definition serves both passes.
2. **Global state**: `__gmpfr_emin/emax/flags` and the default precision/rounding become `__device__ __managed__` (shared host/device).
3. **assert/abort**: `mpfr_assert_fail` (`fprintf+abort`) and `gmp_die` become `__trap()` on the device.
4. `__builtin_unreachable` **miscompile**: enabling `MPFR_HAVE_BUILTIN_UNREACHABLE` makes its `__builtin_constant_p/__builtin_unreachable` path **miscompile**

MPFR_ASSERTN on the device (a provably-true assertion fired). Dropping the define fixes it.

5. **Allocation routing** (the subtle one): in mini-gmp mode `mpfr_allocate_func` (`mpfr-gmp.c`) fetches mini-gmp’s allocation hooks via `mp_get_memory_functions()`—**host function pointers**. Calling them from the device returned garbage (e.g. echoing the size, 0x28). Patched so that under `__CUDA_ARCH__` the allocate/reallocate/free funcs call the device heap directly.
6. **Constant caches**: `mpfr_const_pi/log2/euler/catalan` hold a compute-function **pointer** (a host address) at static init and memoise into a shared mutable global—an illegal instruction / data race on the device. They are rerouted to cacheless wrappers `mpfr_const*_cuda` that do MPFR_SAVE_EXPO (required—else the constants come out wrong) and call `*_internal` directly.

3.3 The hardest problem: device miscompile of `mpfr_div`

The deepest problem was that `mpfr_div`’s **generic path** (**precision ≥ 3 limbs**) **miscompiles on the device and returns inf/garbage** while **the same code is correct on the host**. It cascaded: `const_pi/const_log2` divide internally, so they came out wrong/crashed (the earlier `const_pi`-derived `mpn_sqrtrm p[n-1]==0` assertion was a **downstream symptom**—the broken quotient fed an unnormalised value into the AGM’s `mpfr_sqrt`), which blocked `log`, `sin`, `cos`, `atan`, `large exp`.

Exhaustive bisection confirmed every sub-primitive was **correct standalone on the device** (`mpn_divrem_1`, `mpn_divrem`, mini-gmp mpz division, `count_leading_zeros`, `udiv_qrnd`, MPFR_TMP); `-O0/-G` did not help; and the miscompiled construct resisted statement-level isolation (instrumented writes were reordered/dropped around the calls). Rather than chase the nvcc codegen issue, we **substituted a self-contained, correctly-rounded `mpfr_div` built only on mpz primitives** (`mpfr_div_cuda`). It builds the operand significands as mpz integers, does one `mpz_tdiv_qr` with guard bits, rounds (RNDN/Z/U/D/A) from the round/sticky bits and the remainder, sets the exponent, and `mpfr_check_ranges`. The public `mpfr_div` is `#ifdef __CUDA_ARCH__`-guarded to call it on the device. It matches host `mpfr_div` bit-for-bit, which unblocked all transcendentals (`make test-mpfr-trans`: 128/128 threads match).

3.4 Stack, arena, and parallelism

1. **Stack overflow**: a deep chain such as `mpc_mul` $\rightarrow$ `mpfr_fmms` $\rightarrow$ `mpfr_sub` $\rightarrow$ `mpfr_sub1`, with MPFR’s large stack frames, overflows the default ~ 1 KB CUDA thread stack. Raise it with `cudaDeviceSetLimit(cudaLimitStackSize, 128 KB)`.

2. **Allocation churn:** per-element device `malloc/free` dominates (the real AXPY was $0.9\times$ before). A **per-thread bump arena** (bump-allocate from a fixed slab, free is a no-op, reset per work item) removes this churn.
3. **Parallelism starvation:** the deep chain keeps its limbs in (global-memory) arena slabs, so many resident warps are needed to hide the latency. A small launch starves it (MPC was $0.7\times$ at $N=1024$); a fixed pool of $\sim 8\text{--}16\text{K}$ resident threads grid-striding over the vector hides the latency and gives the $\sim 40\times$ below.

3.5 Coexistence: the `cu_` namespace and the umbrella header

To coexist with the real CPU `libgmp/libmpfr/libmpc` in one binary, every exported symbol is mechanically renamed into a `cu_` namespace (`mpfr_mul`→`cu_mpfr_mul`, etc.; `tools/cu_prefix.py`). Because type names, enum constants, public macros and include guards also clash with the system headers, a single-include **umbrella header** (`mpc_cuda.cuh`) was generated by `tools/make_umbrella.py` with those namespaced to `cu_/CU_`. Thus `#include "mpc_cuda.cuh"` can sit in the **same translation unit** as the system `<gmp.h>/<mpfr.h>/<mpc.h>` with no clash.

3.6 Other bugs and lessons

- **Inline-PTX bug in `mul.c`:** `src/mpfr_cuda/mul.c` had a hand-written PTX `add.ssaaaa(..., 0, t)` passing the integer literal 0 (4 bytes) to the 64-bit "l" constraint, a compile error. Cast to `(mp_limb_t) 0`. This had dropped `mul.o` (`cu_mpfr_mul`) from the archive, breaking relinking of `cu_mpfr/cu_mpc` programs.
- **`cudaLimitStackSize` return value:** on GB10 `cudaLimitStackSize` has a settable maximum (a few hundred KB); a 512KB request is **rejected with `cudaErrorInvalidValue`**. If the return is not checked the stack stays at the $\sim 1\text{KB}$ default and the deep `cu_mpc` chain overflows (observed as “illegal memory access”). 128KB is accepted and sufficient. **Always check the return value of `cudaDeviceSetLimit`.**

4 The Compile-Time Fixed-Precision Fast Path

Separate from the runtime-precision MPFR/MPC port, a fast path was added that **fixes the precision as a compile-time template parameter**, so the significand stays in registers.

4.1 Design

`cu_fp::cu_freal<PB>` (real) and `cu_fp::cu_fcomplex<PB>` (complex) are header-only templates. `PB` is the mantissa width in bits and may be any **multiple of 32** (32, 64, 96, ..., 256, 512, 1024, ...). The significand is held in $\lceil PB/64 \rceil$ 64-bit limbs; because the precision is a compile-time constant the schoolbook multiply is fully unrolled and `ptxas` promotes the local arrays to registers (no global-memory arena round-trips). Rounding is concentrated in a single `finalize()` helper that uniformly handles any precision including the sub-limb case ($SB \in \{0, 32\}$). Operators `+` `-` `*` and `from_double/(double)` conversions are provided. The real mul/add/sub are **bit-identical** to MPFR RNDN (validated for `PB=32–2048`), and the complex multiply is correctly rounded per component—`re` = $\circ(a_r b_r - a_i b_i)$, `im` = $\circ(a_r b_i + a_i b_r)$ —computed from the **exact 2N-limb partial products with a single rounding**, matching MPC’s `MPC_RNDNN`.

4.2 Elementary functions

`cu_fmth.cuh/cu_fcmth.cuh` provide fixed-precision elementary functions: real `cu_{sqrt,cbirt,exp,expm1,log,log1p,sin,cos,tan,atan,sinh,cosh,asin,acos,atanh,pow}` and complex `cu_{cexp,clog,csqrt,csin,ccos,ctan,csinh,ccosh}`. The strategy is a **guard-bit high-accuracy approximation**: everything is computed at a working precision of `PB+128` bits using Newton iteration (`div/sqrt/cbirt`) and argument-reduced series (`exp/log/trig/hyperbolic`), then rounded to `PB`. It is deliberately **not correctly rounded** (the table-maker’s-dilemma cases would need an unbounded Ziv loop, incompatible with register-resident fixed precision). Agreement with MPFR/MPC is ≤ 1 ULP—empirically 0 ULP over the validation range (`PB=64–1024`). Special functions (`gamma`, `zeta`, `erf`, Bessel, `Ai`) are left as future work.

5 Performance Evaluation

The primary platform is the NVIDIA GB10 (compute capability 12.1, CUDA 13, `-fmad=false`); for comparison we report **side by side** the results on an NVIDIA H100 NVL (compute capability 9.0, `sm_90`) in each subsection. Unless noted, precision is 1024 bits and all results are bit-identical to the host (≤ 1 ULP for elementary functions). Because the two hosts use different CPUs (GB10’s Grace ARM vs the H100 box’s x86), the fair hardware-to-hardware metric is the **GPU time in ms**; GPU-vs-CPU speedups are listed for completeness but each is relative to its own host. The head-to-head cross-platform synthesis (over AXPY, linear algebra, elementary functions and the precision sweep) is collected **at**

the end of this section (Section 5.7).

5.1 AXPY (real and complex)

The runtime-precision MPFR/MPC AXPY $y = ax + y$ ($N=16384$, 1024 bits) was compared on GPU and CPU (this library’s host path). The same demo was run on the H100 and placed next to GB10 in Table 2 (the H100 GPU time is about $4\times$ shorter).

表 2 Runtime-precision AXPY ($N=16384$, 1024 bits, all bit-exact). Speedups are each relative to that machine’s own host CPU (GB10: Grace ARM; H100: x86).

Demo	GB10 (sm_121)			H100 (sm_90)	
	GPU	CPU	Speedup	GPU	Speedup
MPFR real	1.17 ms	48.2 ms	$41.3\times$	0.295 ms	$143\times$
MPC complex	4.81 ms	191.7 ms	$39.9\times$	1.037 ms	$160\times$

The speedup required both the bump arena (removing allocation churn) and enough resident threads (hiding memory latency), as discussed in Section 3.

5.2 Basic linear algebra (matrix-vector and matrix multiply)

Matrix-vector $y=A\cdot x$ and matrix multiply $C=A\cdot B$ were implemented for real and complex (Table 3). The dot-product accumulator/scratch are **stack-backed** (MPFR_DECL_INIT; the complex MPC_DECL_INIT built from `mpfr_custom_init_set`), so the bump arena only holds one multiply-add and is reset every inner iteration. Grid-striding over the output elements lets the $O(N^2)$ matrix multiply saturate the device, yielding $> 100\times$. All results bit-exact. The same kernels were run on the H100 and placed next to GB10 in Table 3: the H100 wins the saturating matrix multiply but loses the small- N matrix-vector kernel to occupancy starvation (see Section 5.7).

5.3 Elementary functions

A suite of functions was evaluated over $N=4096$ inputs (GPU vs CPU, every function bit-exact).

- **MPFR:** sqrt cbrt exp expm1 log log1p sin cos tan atan sinh cosh $\Rightarrow$ **42–77** $\times$ on the GPU.
- **MPC:** sqr sqrt exp log sin cos tan sinh cosh asin acos atan $\Rightarrow$ **22–63** $\times$ on the GPU.

表 3 Multiple-precision linear algebra (1024 bits, all bit-exact). Speedups are each relative to that machine’s own host CPU.

Kernel	N	GB10 (sm_121)			H100 (sm_90)	
		GPU	CPU	Speedup	GPU	Speedup
MPFR matrix-vector	256	11.3 ms	177 ms	15.6×	15.80 ms	9.2×
MPFR matrix multiply	96	23.0 ms	2.41 s	105 ×	12.00 ms	175 ×
MPC matrix-vector	128	24.4 ms	179 ms	7.3×	29.68 ms	5.2×
MPC matrix multiply	64	27.9 ms	2.88 s	103 ×	15.63 ms	155 ×

The per-function breakdown is shown side by side in Table 4 and Figure 1: the **H100 per-function measurements** (GPU vs its host CPU) are given explicitly, while the **GB10 results are overlaid as a shaded min–max band** (the ranges above: MPFR 42–77×, MPC 22–63×; the per-function GB10 timings are not re-measurable on this machine). Both show the same shape with a wide spread: the cheap kernels (`sqrt`, `cbrt`, `sqr`) reach 170–217× because the GPU pass is short while the host MPFR/MPC cost is dominated by setup, whereas the heavy multi-AGM kernels (`log`, complex `asin/acos`) sit at the low end. (The GB10 and H100 speedups are each measured against that machine’s own host CPU and so are not directly comparable as ratios.)

表 4 Elementary/transcendental GPU speedups ($N=4096$, 1024 bits, all bit-exact). H100 is the per-function measurement; GB10 is the range over the same functions (MPFR 42–77×, MPC 22–63×).

MPFR (real)				MPC (complex)			
func	GPU	CPU	speedup	func	GPU	CPU	speedup
<code>sqrt</code>	0.86	185.9	217×	<code>sqr</code>	0.24	42.6	180×
<code>cbrt</code>	3.51	597.7	170×	<code>sqrt</code>	2.93	362.6	124×
<code>exp</code>	10.60	1271.3	120×	<code>exp</code>	23.71	3255.8	137×
<code>expm1</code>	9.69	1305.7	135×	<code>log</code>	77.46	6378.9	82×
<code>log</code>	39.71	4189.5	105×	<code>sin</code>	24.79	1398.0	56×
<code>log1p</code>	25.99	3510.6	135×	<code>cos</code>	24.75	1396.7	56×
<code>sin</code>	3.14	362.1	115×	<code>tan</code>	36.80	1583.7	43×
<code>cos</code>	2.05	219.4	107×	<code>sinh</code>	24.66	1422.3	58×
<code>tan</code>	3.37	380.3	113×	<code>cosh</code>	24.61	1420.5	58×
<code>atan</code>	21.75	1834.1	84×	<code>asin</code>	87.69	6886.2	79×
<code>sinh</code>	20.75	957.1	46×	<code>acos</code>	94.98	7511.9	79×
<code>cosh</code>	14.32	936.4	65×	<code>atan</code>	121.12	11179.5	92×

Times in ms. For reference, the GB10 speedup ranges over the same functions are MPFR 42–77×, MPC 22–63× (shown as the shaded band in Figure 1).

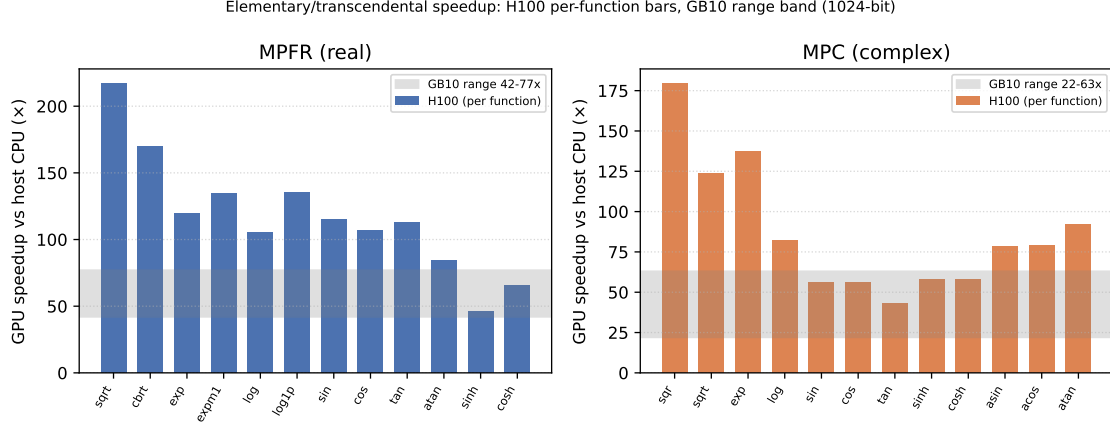


Figure 1: Elementary/transcendental GPU speedup over the host CPU. Bars = H100 per-function measurement; grey band = GB10 range (MPFR 42–77 $\times$, MPC 22–63 $\times$). Left: MPFR real; right: MPC complex. 1024-bit, all bit-exact.

5.4 The fixed-precision fast path and a precision sweep

The fixed-precision path is far faster than the runtime path. For 1024-bit AXPY ($N=2^{20}$), `cu.freal` ran **142 $\times$** faster than the existing `cu_mprf`-GPU path and **352 $\times$** faster than system CPU MPFR (both bit-exact).

To expose the precision dependence, AXPY was swept with `make bench3`. GB10 was measured over 128–8192 bits (Table 5); the H100 sweep was **extended to 128–65536 bits** so the fixed, runtime and CPU paths fully cross (Table 6). The GB10 and H100 curves are overlaid in Figure 2 (real), Figure 3 (complex) and Figure 4 (speedup); solid = H100, dashed = GB10.

On GB10 the `cu.freal` time is flat through 1024 bits (fully register-resident), **jumps $\sim 7\times$ at 2048 bits** (where the $N = 32$ -limb significand spills from registers to local memory), and reaches parity with one CPU core at 8192 bits. The complex side has the same shape with an even larger lead (peak $\sim 48\times$ vs `cu_mpc` and $\sim 107\times$ vs CPU at 1024 bits).

On the H100 the larger machine hides latency better, so two effects appear that are consistent with the occupancy story of Section 5.7 (Table 6). First, the **GPU runtime path** (`cu_mprf/cu_mpc`) is **~ 2 – $5\times$ faster on the H100 than GB10 at every precision** (e.g. 1024-bit real: 0.120ms vs GB10’s 0.388ms). Second, the H100’s **fixed-precision spill knee at 2048 bits is far softer**—`cu.freal` rises only $\sim 2.75\times$ from 1024 to 2048 bits (0.008 $\rightarrow$ 0.022ms) versus GB10’s $\sim 7\times$ jump (0.008 $\rightarrow$ 0.056ms)—because the H100’s larger register file and local-memory bandwidth cushion the register $\rightarrow$ local spill.

表 5 Precision sweep of the fixed-precision path (AXPY, N=4096, GB10, all bit-exact)

Bits	cu_freal	cu_mpfr	CPU	freal/cu_mpfr	freal/CPU
128	0.005	0.050	0.190	10.2×	38.5×
256	0.005	0.088	0.222	19.6×	49.1×
512	0.006	0.174	0.320	29.5×	54.1×
1024	0.008	0.388	0.677	47.4×	82.6×
2048	0.056	0.506	0.322	9.0×	5.7×
4096	0.184	0.996	0.473	5.4×	2.6×
8192	0.823	1.989	0.814	2.4×	1.0×

Times in ms (8-iteration average).

■All paths cross (H100). Pushing the precision further, the fixed-precision advantage disappears entirely. For real AXPY, `cu_freal` reaches **parity with cu_mpfr at 8192 bits** (0.430 vs 0.436 ms) and **crosses the CPU at 16384 bits** (2.477 vs 2.847 ms), then degrades catastrophically (64.6 ms at 32768, 482 ms at 65536) as the spilled schoolbook $O(n^2)$ thrashes local memory. The complex side has the same shape: `cu_fcomplex` crosses `cu_mpc` at 16384 bits and the CPU before 32768 bits. Meanwhile the **GPU runtime path stays faster than the CPU even at 65536 bits** (real 3.6 vs 6.4 ms, complex 31.4 vs 60.9 ms), showing that at high precision the runtime path (or the sub-quadratic CPU) is preferable to the fixed path. Every sweep point is bit-identical to the host (`rel=0`).

表 6 Precision sweep on the H100 (AXPY, N=4096, all bit-exact). GB10 is not measurable here, so see Table 5 for GB10 to 8192 bits. Times in ms (8-iteration average).

Bits	real (MPFR)			complex (MPC)		
	cu_freal	cu_mpfr	CPU	cu_fcplx	cu_mpc	CPU
128	0.004	0.028	0.681	0.009	0.074	1.955
256	0.004	0.036	0.663	0.010	0.100	2.029
512	0.005	0.059	0.776	0.015	0.163	2.595
1024	0.008	0.120	0.786	0.044	0.351	4.670
2048	0.022	0.129	0.888	0.125	0.932	6.309
4096	0.083	0.228	1.235	0.509	1.785	8.403
8192	0.430	0.436	1.787	2.025	3.626	13.105
16384	2.477	0.884	2.847	11.693	7.529	23.115
32768	64.635	1.769	3.443	263.499	15.429	30.636
65536	482.420	3.601	6.375	1929.433	31.360	60.912

Crossover points: real `cu_freal` $\approx$ `cu_mpfr` at 8192 bits and `cu_freal` $>$ CPU at 16384 bits; complex `cu_fcplx` $\approx$ `cu_mpc` at 16384 bits.

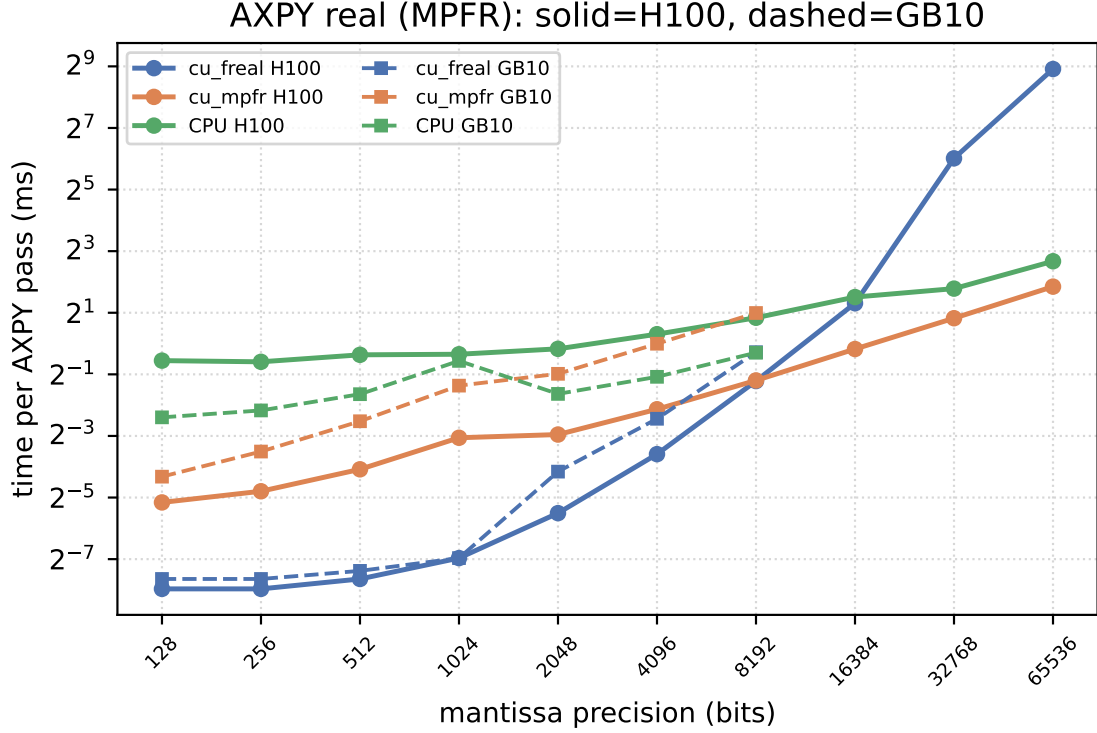


Figure 2: Real AXPY time vs precision (log-log). Solid = H100 (to 65536 bits), dashed = GB10 (to 8192 bits); blue = `cu_freah`, orange = `cu_mpfr`, green = CPU. Fixed precision is flat through 1024 bits, jumps at 2048 (spill), and on the H100 crosses `cu_mpfr` at 8192 bits and the CPU at 16384 bits, then diverges sharply.

5.5 The fixed-precision path on the CPU

`cu_freah/cu_fcomplex` are header-only and compile/run on the host with g++ alone (using `__int128`). On the CPU (single thread) versus system MPFR/MPC, they are **faster only at low precision** (Table 7). `cu_freah`'s multiply is schoolbook $O(n^2)$, whereas GMP/MPFR is sub-quadratic (Karatsuba/Toom/FFT); the GPU won because register-residency and massive parallelism hid the $O(n^2)$ cost, an advantage a single CPU thread lacks, so above ~ 512 bits the algorithmic gap shows.

5.6 Discussion

On the GPU the register-resident fixed-precision path **wins decisively up to ~ 1024 bits** (up to $\sim 83\times$ vs CPU, $\sim 142\times$ vs `cu_mpfr`-GPU), hits a spill knee at 2048 bits, and reaches CPU parity at 8192 bits. On a single CPU thread the same path beats MPFR/MPC only at 128–256 bits. Hence the fixed-precision schoolbook advantage is bounded by (i) **GPU parallelism**

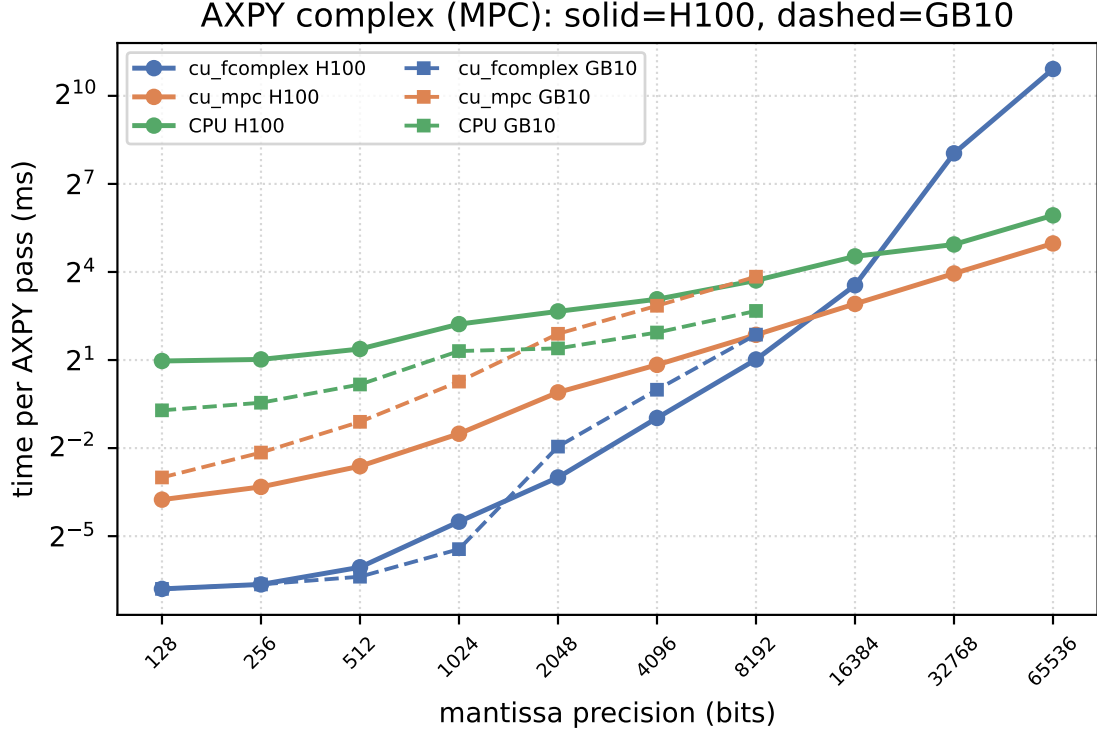


Figure 3: Complex AXPY time vs precision (log-log); series as in Figure 2. Same shape with a larger fixed-precision lead, but on the H100 `cu_fcomplex` crosses `cu_mpc` at 16384 bits and the CPU before 32768 bits.

and (ii) **low/medium precision**; for high precision or on the CPU, MPFR/MPC with its sub-quadratic multiply is preferable.

5.7 Cross-platform comparison: GB10 vs H100 (synthesis)

Finally we collect, from the two-GPU viewpoint, the results reported side by side throughout this section. The six runtime-precision demos (real/complex AXPY, matrix–vector and matrix multiply) were run on both machines from the same source at the same problem sizes and 1024-bit precision, all bit-exact. Because the two hosts use different CPUs (GB10’s Grace ARM vs the H100 box’s x86), the **GPU time in ms** is the fair hardware-to-hardware metric; the GPU-vs-CPU speedups are listed for completeness but each is relative to its own host (Table 8, Figure 6).

The pattern is consistent and explainable by occupancy. The H100 dominates the **high-parallelism** kernels—AXPY ($\sim 4\times$, memory-bandwidth bound over 16384 elements) and matrix multiply ($\sim 1.8\text{--}1.9\times$, an $O(N^2)$ grid that saturates the device)—where its 132 SMs and HBM3 bandwidth are fully used. It **loses** the matrix–vector kernels ($0.72\text{--}0.82\times$): with one

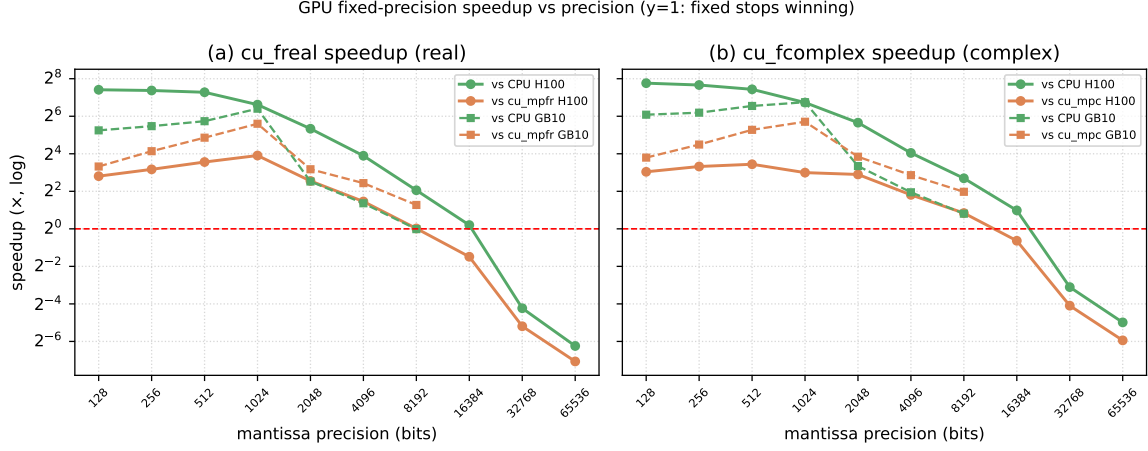


图 4 GPU fixed-precision speedup vs precision (log-log, $y = 1$ is parity). Solid = H100, dashed = GB10. (a) real (vs CPU and vs cu_mpf); (b) complex (vs CPU and vs cu_mpc). Peak at 1024 bits, spill knee at 2048 bits, and $y < 1$ at high precision where fixed precision loses.

表 7 CPU-only fixed precision vs system MPFR/MPC (AXPY, $N=4096$, single thread; ratio > 1 means fixed precision is faster)

Bits	freal/MPFR	fcplx/MPC
128	1.28 $\times$	1.79 $\times$
256	1.23 $\times$	1.48 $\times$
512	1.03 $\times$	0.99 $\times$
1024	0.71 $\times$	0.67 $\times$
2048	0.09 $\times$	0.17 $\times$
4096	0.03 $\times$	0.06 $\times$
8192	0.01 $\times$	0.03 $\times$

thread per output row and $N = 128$ – 256 , only ~ 128 – 256 threads are ever active, so the launch is **occupancy-starved** and the result is bound by the single-thread latency of one long multiple-precision dot product, where the newer GB10 core (`sm_121`, higher clock) is faster. Figure 7 shows the GPU-vs-CPU speedups on the two platforms side by side.

The **elementary functions** (Table 4, Figure 1) show the same picture: the $N = 4096$ evaluation fills the device, so both machines are fast—the H100 reaches **43–217** $\times$ per function (vs its x86 host) and the GB10 spans **42–77** $\times$ (MPFR) / **22–63** $\times$ (MPC, vs the Grace ARM). The speedups are each relative to their own host and so not directly comparable, but both machines peak on the cheap kernels (`sqrt`, `cbrt`, `sqr`) and bottom out on the heavy multi-AGM kernels (`log`, complex `asin/acos`).

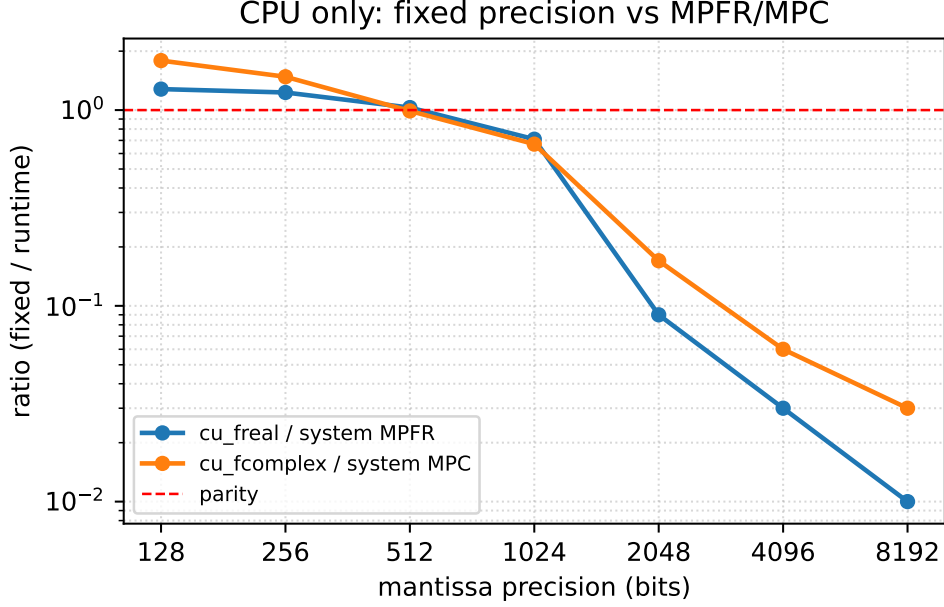


图 5 CPU-only fixed precision vs MPFR/MPC ratio ($\log y$, $y = 1$ is parity). Fixed precision wins only at 128–256 bits; at high precision MPFR/MPC with its sub-quadratic multiply wins.

表 8 GB10 vs H100, runtime-precision demos (1024 bits, all bit-exact). The ratio is GB10/H100 GPU time ($> 1 \Rightarrow$ H100 faster).

Demo	N	GB10 GPU	H100 GPU	H100/GB10
AXPY MPFR	16384	1.17 ms	0.295 ms	3.97 $\times$
AXPY MPC	16384	4.81 ms	1.037 ms	4.64 $\times$
matvec MPFR	256	11.3 ms	15.80 ms	0.72 $\times$
matvec MPC	128	24.4 ms	29.68 ms	0.82 $\times$
matmul MPFR	96	23.0 ms	12.00 ms	1.92 $\times$
matmul MPC	64	27.9 ms	15.63 ms	1.79 $\times$

The **precision sweep** (Section 5.4) is consistent with the same occupancy effect: at $N = 4096$ the H100 hides latency better, so its runtime path is $\sim 2\text{--}5\times$ faster at every precision and its fixed-precision spill knee at 2048 bits is softer ($\sim 2.75\times$ vs GB10’s $\sim 7\times$).

In short, the H100 is the better device when there is enough parallelism to fill it (high-parallelism AXPY, matrix multiply, elementary functions, large- N sweep), while the GB10 is competitive in the latency-bound small- N regime (matrix-vector).

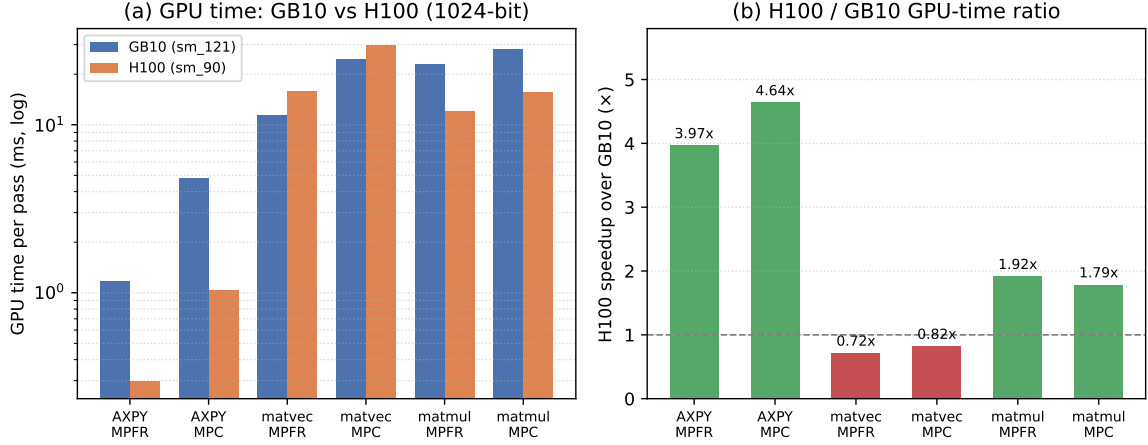


Figure 6 GB10 vs H100. (a) GPU time per pass (log scale); (b) H100 speedup over GB10 ($y = 1$ is parity). The H100 wins the high-parallelism kernels but loses the small- N matvec.

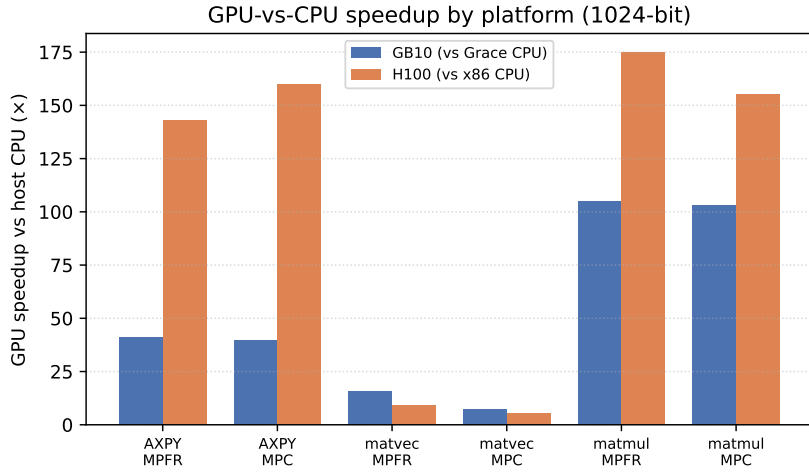


Figure 7 GPU-vs-CPU speedup by platform (each GPU against its own host CPU), 1024-bit. The H100's higher AXPY/matmul speedups partly reflect its faster GPU and partly its different (x86) host baseline.

6 Conclusion and Future Work

GMP(mini-gmp)/MPFR/MPC were **faithfully** ported into CUDA kernels via re-runnable transform scripts, and real (MPFR) and complex (MPC) AXPY were accelerated by $\sim 40\times$ while remaining bit-identical to the CPU. The crux of the port was making it compile (NORETURN/NO_ASM/realloc/_Complex), making it run on the device (RODATA/managed globals/allocation routing/constant caches), and substituting an `mpz`-based `mpfr_div` for the device

miscompile. Stack limits, the bump arena, parallelism, and coexistence (the `cu_` namespace and umbrella) were also addressed.

In addition, a compile-time fixed-precision, register-resident `cu_frexp<PB>/cu_fcomplex<PB>` and fixed-precision elementary functions were introduced. These beat MPFR/MPC by orders of magnitude at low/medium precision on the GPU, and the precision sweep quantified the crossover: **2048 bits is the spill knee, and CPU parity is reached at 8192 bits**. A cross-platform comparison (GB10 vs H100 NVL) further showed that the runtime-precision kernels are **occupancy-bound**: the H100 is $\sim 4\times$ (AXPY) and $\sim 1.8\text{--}1.9\times$ (matmul) faster where there is enough parallelism to fill its 132 SMs, but the small- N matrix-vector kernels stay latency-bound and slightly favour the GB10.

Future work includes fixed-precision implementations of the MPFR/MPC special functions (gamma, zeta, erf, Bessel, ...), reducing register pressure and compile time at high precision, adding sub-quadratic multiplication (Karatsuba, etc.) to the fixed-precision path, and fully race-free per-thread exponent/flag state.

Reproduction: `./configure && make; make check` (minigmp/mpfr/mpc/trans bit-exactness), `make demos/matrix/bench` (performance), `make sample-fixed/check-fixed/check-fmath/bench` (fixed precision). The CPU-reference targets (bench3, coexist, cputest, cpubench) need a system GMP/MPFR/MPC; if it lives in a non-standard prefix, pass `./configure --with-mp-prefix=DIR` (e.g. `~/local`), which adds its `include/lib` (with `rpath`) to the build. The target GPU is selected with `--with-cuda-arch=ARCH` (default `sm_90`; the GB10 runs used `sm_121`). See the user manuals `doc/manual.md` (English) / `doc/manual.ja.md` (Japanese) for details.